PROYECTO INTEGRADOR

PrestaLab

Sistema de reserva y control de préstamo de equipo y espacios de laboratorio

ENTREGA 5 — MVP Backend I

Carrera	Ingeniería en Software
Materia	Arquitectura Orientada a Servicios
Docente	Ing. Luis Leonardo Mendoza Reséndiz
Equipo	Tranquilo nos la navegamos
Integrantes	Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras
Periodo / Semana	Septiembre – Diciembre 2026 · Semana 5 de 8
Repositorio	https://github.com/JhonaRaida/prestalab · tag: entrega-5

Reporte breve de la entrega

Entrega	5 — MVP Backend I
Nombre del proyecto	PrestaLab
Integrantes	Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras
Objetivo de la semana	Demostrar que la arquitectura de las entregas 2 a 4 funciona como software: backend organizado por capas, conectado a PostgreSQL, con CRUD completos, validaciones y manejo de errores 404 y 500.
Qué se implementó	Backend en Node.js + Express con un API Gateway y 5 servicios, cada uno organizado en rutas, controladores, servicios y acceso a datos. CRUD completo de Recursos y Reservas contra PostgreSQL, validaciones (400), manejo de 404 y 500, colección de Postman con 61 peticiones y README de ejecución.
Qué cambió respecto a la entrega anterior	La arquitectura y el modelo de datos de la Entrega 4 no cambiaron. La colección de Postman ahora inicia sesión sola, la documentación se reorganizó (README solo con instalación y ejecución; la parte técnica en docs/) y el contrato OpenAPI dejó de marcarse como borrador. Detalle en la sección 4.
Problemas encontrados	(1) Al abrir la carpeta postman/ como carpeta local, Postman convirtió la colección a su formato y borró los .json del repositorio. (2) Al probar petición por petición salían errores 401, porque cada una usa el token de una cuenta distinta. (3) La consigna pide CRUD completo, pero el diseño no permite borrar historial.
Cómo se resolvieron	(1) Se recuperaron los archivos con git restore; la colección se importa como archivos (File → Import) o se ejecuta con Newman. (2) La colección inicia sesión sola cuando falta el token o ya venció. (3) «Eliminar» es una baja lógica: DELETE pone un recurso en baja y cancelar una reserva cambia su estado (sección 2.1).
Pendientes para la siguiente semana	Documentar con evidencias la autenticación con JWT, los roles, las reglas de negocio y la operación que involucra varios servicios (reserva → préstamo → incidencia).
Repositorio y versión	https://github.com/JhonaRaida/prestalab · tag: entrega-5

1. Estructura del backend

El backend sigue la arquitectura definida en la Entrega 2: un **API Gateway** como única entrada pública y **cinco servicios** independientes. Cada servicio que guarda datos tiene su propia base en PostgreSQL y ningún servicio lee la base de otro.

Componente	Puerto	Base de datos	Recursos que expone
Gateway	8080	—	Entrada única: valida el JWT, enruta y limita peticiones
Usuarios	3001	usuarios_db	sesiones, usuarios, sanciones
Inventario	3002	inventario_db	laboratorios, categorías, recursos
Reservas	3003	reservas_db	disponibilidad, reservas, préstamos, incidencias
Notificaciones	3004	notificaciones_db	notificaciones
Reportes	3005	— (lee de otros servicios)	reportes de uso, ocupación y retrasos

1.1 Capas de cada servicio

Ningún servicio concentra su código en un solo archivo. Todos tienen la misma estructura de cuatro capas, más el arranque:

Capa	Archivo	Responsabilidad
Rutas	routes.js	Qué URL atiende y qué rol puede usarla
Controlador	controller.js	Valida la entrada (Zod) y arma la respuesta HTTP
Servicio	service.js	Reglas de negocio (traslapes, plazos, permisos por laboratorio)
Acceso a datos	repository.js	Consultas SQL parametrizadas contra PostgreSQL
Arranque	app.js · index.js	Monta Express, middlewares y manejadores de error

El código común está en `shared/`: `config.js` (variables de entorno), `db.js` (conexión a PostgreSQL), `errors.js` (formato de error y manejadores 404 y 500), `validar.js`, `auth.js` y `http.js` (llamadas entre servicios).

1.2 Variables de entorno

Toda la configuración sensible se lee de un archivo `.env` que **no se sube al repositorio** (está en `.gitignore`). El repositorio solo incluye `.env.example` con valores de ejemplo:

Variable	Para qué sirve
JWT_SECRET	Firma de los tokens de sesión
SERVICE_TOKEN	Token interno: los servicios solo aceptan peticiones que vienen del Gateway
JWT_EXPIRA_SEGUNDOS	Duración del token de sesión (opcional, 3600 = 1 hora)

Variable	Para qué sirve
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD	Conexión a PostgreSQL
PORT_GATEWAY … PORT_REPORTES	Puertos del Gateway y de cada servicio
TOLERANCIA_MINUTOS, SANCION_DIAS, …	Reglas de negocio ajustables (opcionales)

1.3 Conexión con la base de datos y ejecución

Cada servicio se conecta a su base con el paquete pg y consultas parametrizadas, sin ORM. Al arrancar, si las tablas están vacías, se cargan solos los datos de prueba de la Entrega 4. El sistema se ejecuta siguiendo el README:

```
git clone https://github.com/JhonaRaida/prestalab.git
cd prestalab
cp .env.example .env        # y cambiar JWT_SECRET y SERVICE_TOKEN
docker compose up --build    # listo cuando aparece "[gateway] escuchando en el puerto 8080"
```

Con el sistema arriba, `GET /health` confirma que el Gateway y los cinco servicios responden:

```
GET http://localhost:8080/health
{
  "gateway": "ok",
  "servicios": {
    "usuarios": "ok",
    "inventario": "ok",
    "reservas": "ok",
    "notificaciones": "ok",
    "reportes": "ok"
  }
}
```

2. CRUD completos

La consigna pide el CRUD completo de al menos dos recursos principales. Se eligieron **Recursos** (el equipo y los espacios que se prestan) y **Reservas** (el apartado que hace el alumno o docente), porque son el centro del problema que resuelve PrestaLab.

Operación	Recursos (Inventario)	Reservas
Crear	E-14 · POST /recursos	E-20 · POST /reservas
Consultar varios	E-13 · GET /recursos (filtros y paginación)	E-21 · GET /reservas (historial con filtros)
Consultar uno	E-15 · GET /recursos/{id}	E-22 · GET /reservas/{id}
Modificar	E-16 · PUT /recursos/{id} · E-17 · PATCH (estado)	E-23 · PATCH /reservas/{id} (horario)
Eliminar	E-18 · DELETE /recursos/{id} → estado «baja»	E-23 · PATCH /reservas/{id} → estado «cancelada»

2.1 Por qué «eliminar» es una baja lógica

En la Entrega 3 se definió que DELETE da de baja (baja lógica), y en la Entrega 4 se decidió que **no se pierde el historial**: un recurso dado de baja o una reserva cancelada siguen existiendo para los préstamos, incidencias y reportes que dependen de ellos. Por eso:

- **Recursos:** `DELETE /recursos/{id}` responde 204 y deja el recurso en estado `baja`. Ya no se puede reservar, pero su historial sigue disponible.
- **Reservas:** se cancela con `PATCH /reservas/{id}` y `{"estado":"cancelada"}`, como quedó en el contrato (E-23). La reserva libera el horario y se conserva en el historial del usuario.

Borrar físicamente estos registros rompería los reportes (RF-10) y el control de sanciones (RF-07).

2.2 Validaciones y manejo de errores

Los controladores validan la entrada con **Zod** antes de llegar a la lógica de negocio. Todos los errores usan el mismo formato definido en la Entrega 3 (`codigo`, `mensaje`, `ruta`, `fecha`, `idPeticion` y, si aplica, `detalles` por campo).

El manejador de 500 **no revela detalles internos** al cliente: el detalle real se escribe en el log del servidor junto con el `idPeticion`, para poder rastrearlo.

Situación	HTTP	Código	Dónde se maneja
Datos faltantes o con tipo incorrecto	400	VALIDACION_FALLIDA	Controlador (Zod)
JSON mal formado	400	JSON_INVALIDO	shared/errors.js
Registro con ese id no existe	404	NO_ENCONTRADO	Servicio → Errores.noEncontrado()
Ruta que no existe	404	NO_ENCONTRADO	shared/errors.js · rutaNoEncontrada

Situación	HTTP	Código	Dónde se maneja
Error inesperado (p. ej. la BD no responde)	500	ERROR_INTERNO	shared/errors.js · manejadorErrores

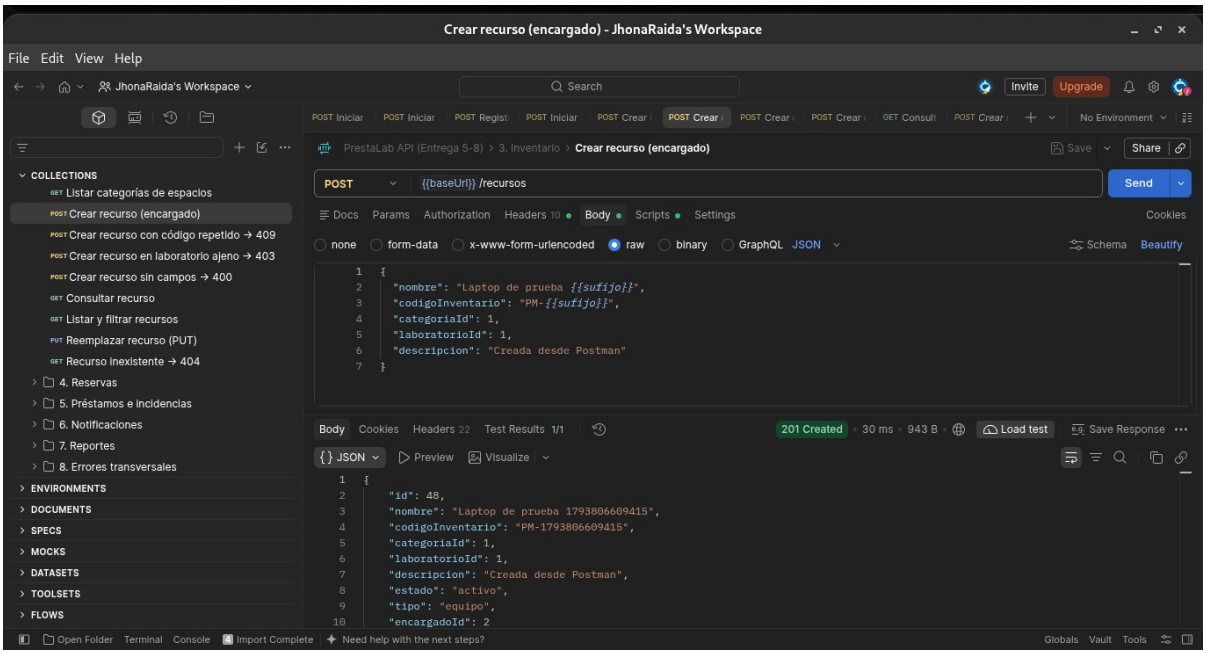
3. Evidencias

Las evidencias se obtuvieron el 4 de noviembre de 2026 con el sistema corriendo contra PostgreSQL. El CRUD de **Recursos** se muestra con capturas de Postman (cuenta de encargado). El de **Reservas** y los errores se muestran con la petición enviada al Gateway y la respuesta real (cuenta **alumno1**); en las listas aparece solo el primer elemento. Todas las respuestas están completas en `entregas/entrega-5/evidencias/` y se pueden repetir con `npm run evidencias`.

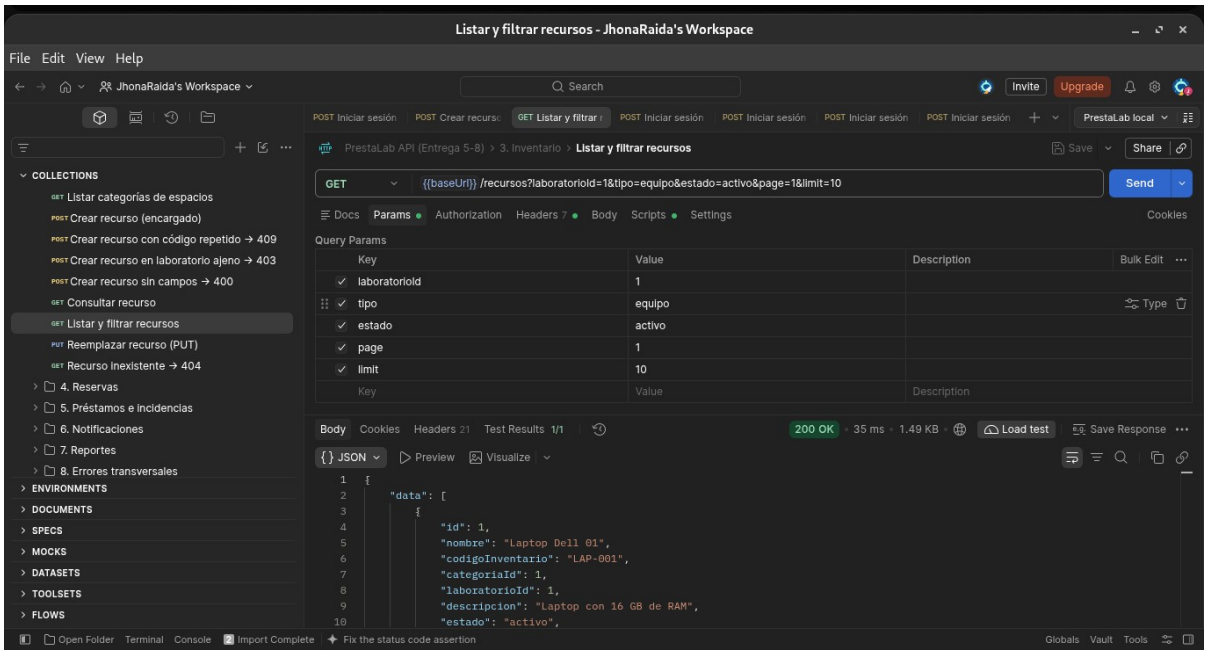
Evidencia pedida	Recursos	Reservas
Crear un registro	Captura 1	V1
Consultar uno y varios	Capturas 3 y 2	V3 · V2
Modificar un registro	Captura 4	V4
Eliminar un registro	Captura 5	V5 · V6
Error de validación	Captura 6	V7
Recurso inexistente	Captura 7	V8

3.1 CRUD de Recursos (Postman)

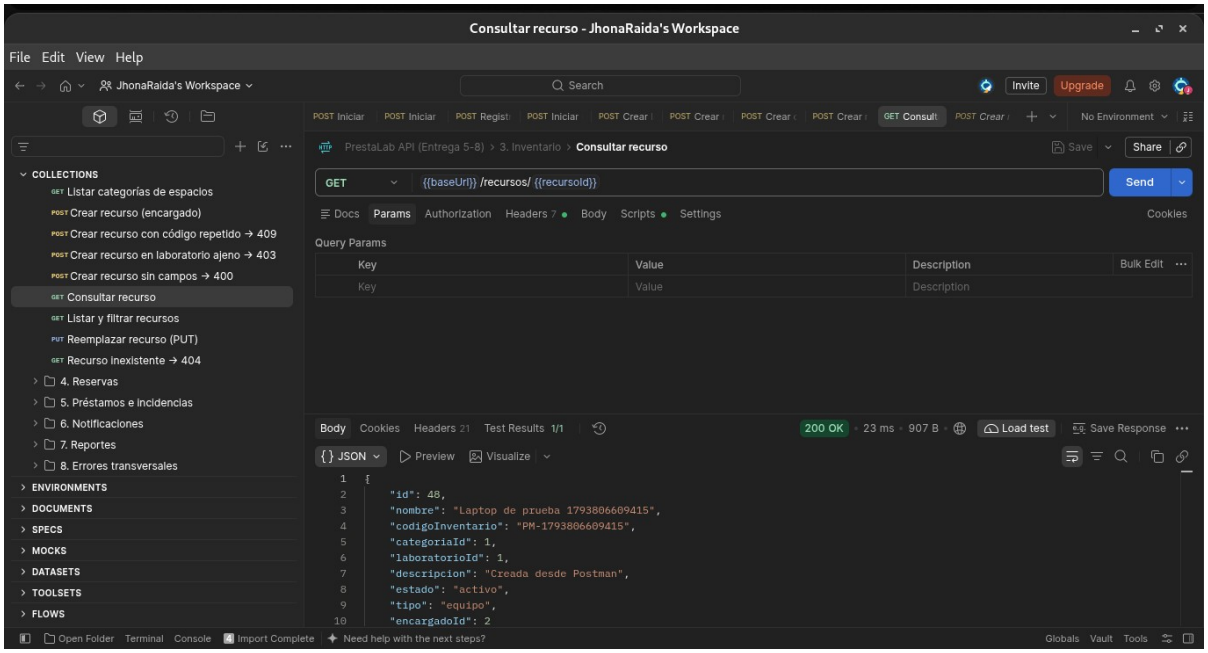
El encargado da de alta una laptop (id 48) y sobre ese mismo registro se hacen la consulta, la modificación y la baja, así que todas las capturas muestran datos guardados en PostgreSQL.



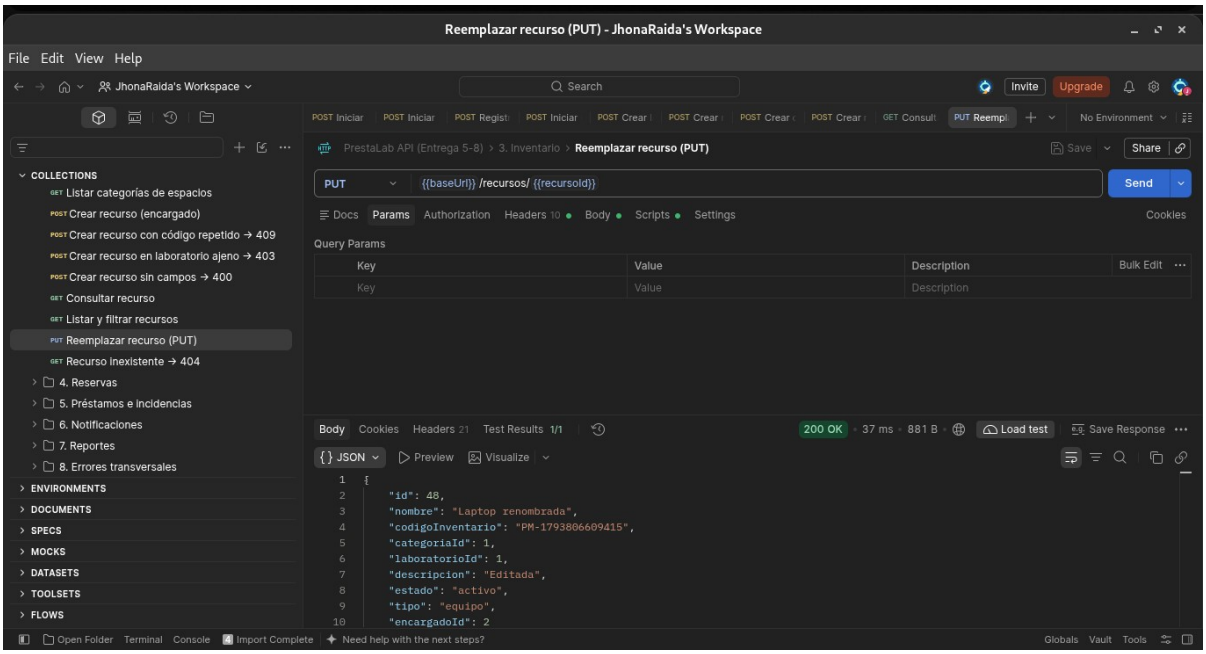
Captura 1. POST /recursos: 201 Created con el recurso guardado (id 48, estado «activo»).



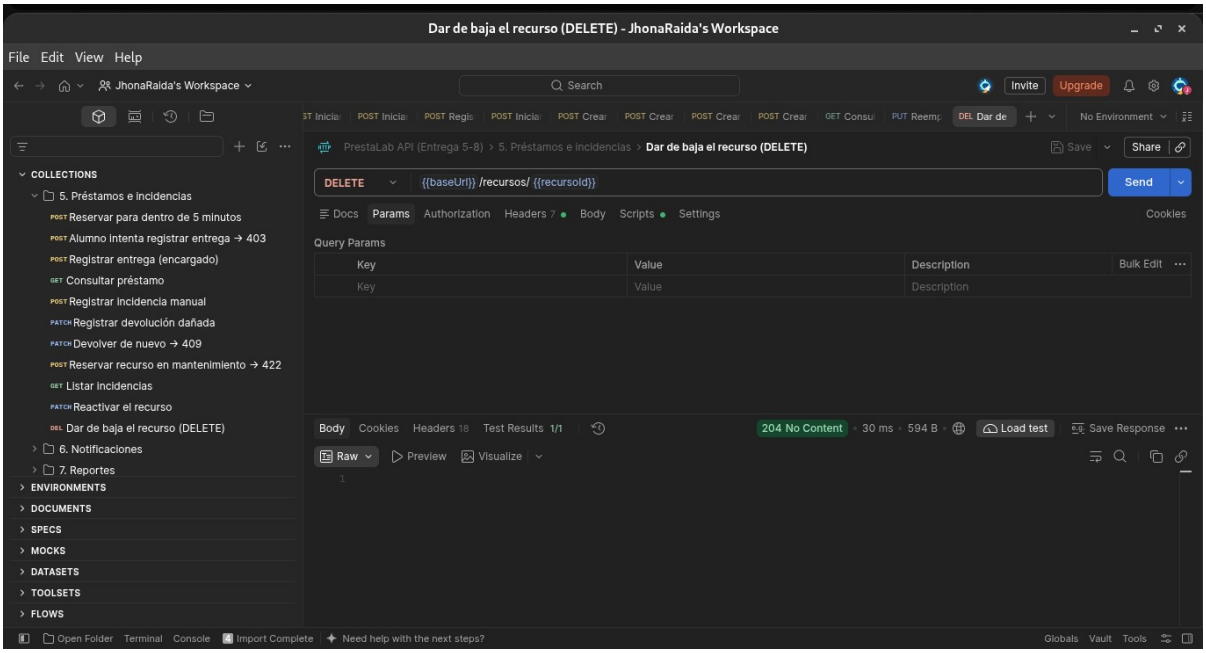
Captura 2. GET /recursos con filtros (laboratorio, tipo y estado) y paginación: 200 OK con la lista en «data».



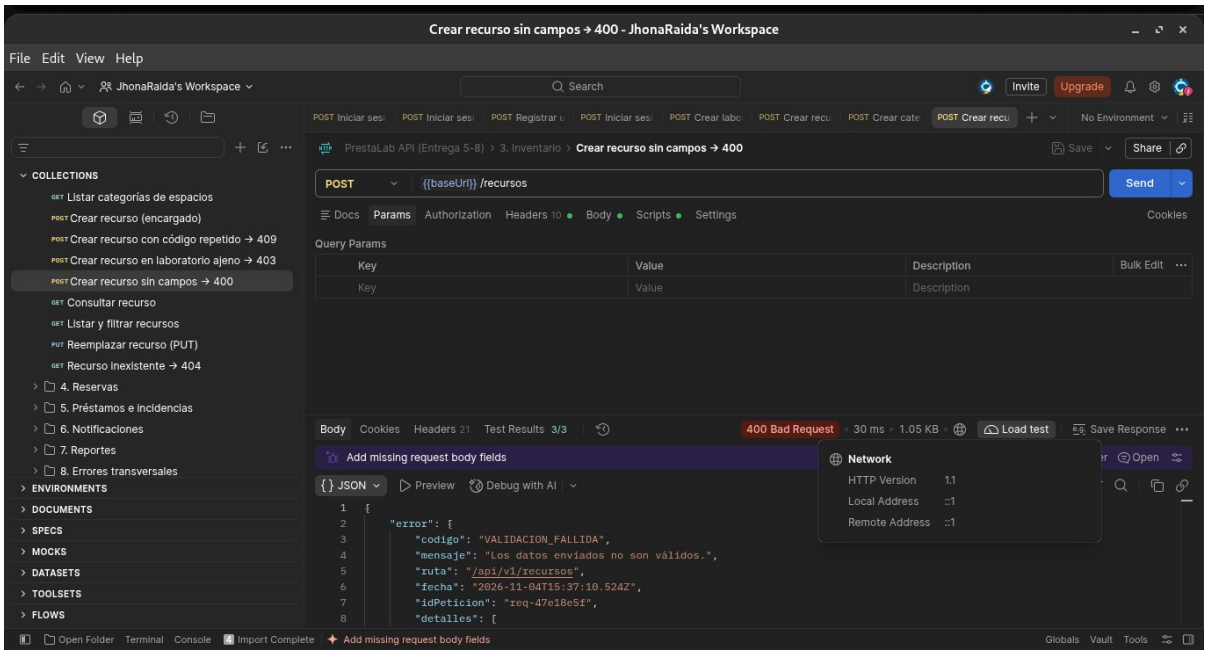
Captura 3. GET /recursos/48: 200 OK con los datos del recurso creado.



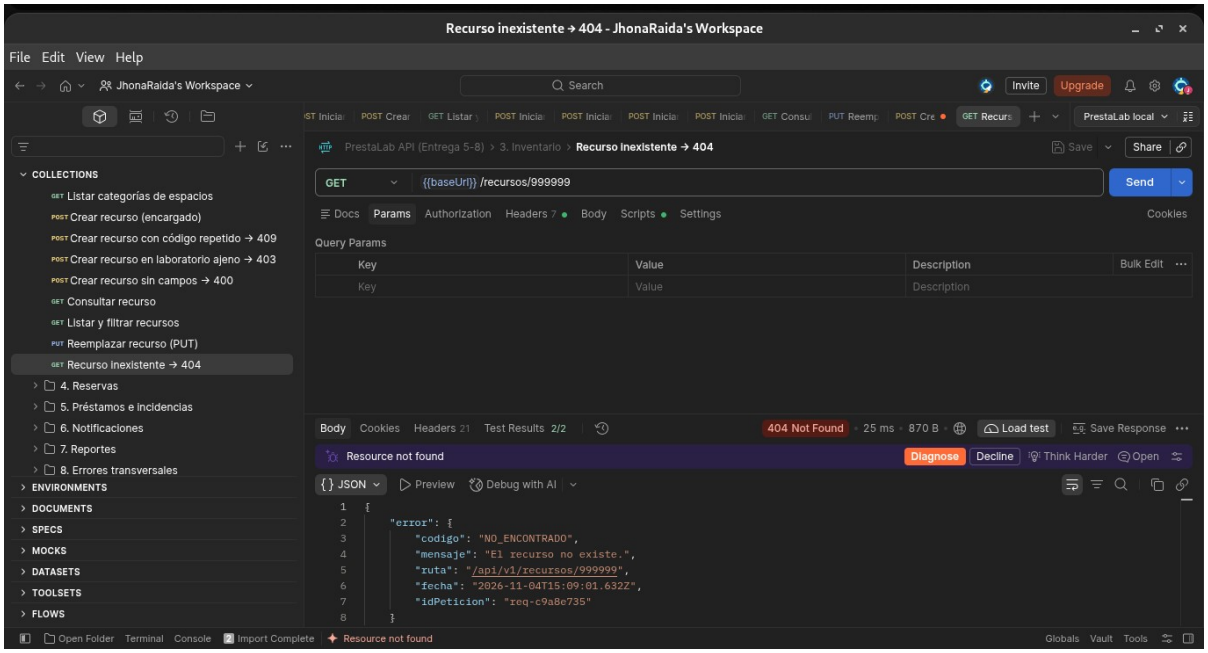
Captura 4. PUT /recursos/48: 200 OK; el nombre cambia a «Laptop renombrada» y la descripción a «Editada».



Captura 5. DELETE /recursos/48: 204 No Content. Es una baja lógica: el recurso queda en estado «baja».



Captura 6. POST /recursos solo con { "nombre": "x" }: 400 VALIDACION_FALLIDA con el detalle de cada campo.



Captura 7. GET /recursos/999999: 404 NO_ENCONTRADO, «El recurso no existe.»

3.2 CRUD de Reservas

V1. Crear una reserva

El alumno reserva un recurso libre. Antes, el sistema comprobó la disponibilidad del horario.

Petición

```
POST /api/v1/reservas
{
  "recursoId": 1,
  "inicio": "2026-11-07T16:00:00.000Z",
  "fin": "2026-11-07T17:00:00.000Z"
}
```

Respuesta · HTTP 201

```
{
  "id": 11,
  "usuarioId": 4,
  "recursoId": 1,
  "inicio": "2026-11-07T16:00:00.000Z",
  "fin": "2026-11-07T17:00:00.000Z",
  "estado": "confirmada",
  "creadaEn": "2026-11-04T16:03:05.872Z"
}
```

V2. Consultar varias reservas (historial)

Petición

```
GET /api/v1/reservas?page=1&limit=3
```

Respuesta · HTTP 200

```
{
  "data": [
    {
      "id": 11,
      "usuarioId": 4,
      "recursoId": 1,
      "inicio": "2026-11-07T16:00:00.000Z",
      "fin": "2026-11-07T17:00:00.000Z",
      "estado": "confirmada",
      "creadaEn": "2026-11-04T16:03:05.872Z"
    },
    // … 2 elemento(s) más
  ],
  "meta": {
    "page": 1,
    "limit": 3,
    "total": 3
  }
}
```

V3. Consultar una reserva

Petición

```
GET /api/v1/reservas/11
```

Respuesta · HTTP 200

```
{
  "id": 11,
  "usuarioId": 4,
  "recursoId": 1,
  "inicio": "2026-11-07T16:00:00.000Z",
  "fin": "2026-11-07T17:00:00.000Z",
  "estado": "confirmada",
  "creadaEn": "2026-11-04T16:03:05.872Z"
}
```

V4. Modificar el horario

Se mueve la reserva a otro día y se amplía a dos horas.

Petición

```
PATCH /api/v1/reservas/11
{
  "inicio": "2026-11-08T16:00:00.000Z",
  "fin": "2026-11-08T18:00:00.000Z"
}
```

Respuesta · HTTP 200

```
{
  "id": 11,
  "usuarioId": 4,
  "recursoId": 1,
  "inicio": "2026-11-08T16:00:00.000Z",
  "fin": "2026-11-08T18:00:00.000Z",
  "estado": "confirmada",
  "creadaEn": "2026-11-04T16:03:05.872Z"
}
```

V5. Eliminar (cancelar) la reserva

Cancelar es la eliminación lógica de una reserva.

Petición

```
PATCH /api/v1/reservas/11
{
  "estado": "cancelada"
}
```

Respuesta · HTTP 200

```
{
  "id": 11,
  "usuarioId": 4,
  "recursoId": 1,
  "inicio": "2026-11-08T16:00:00.000Z",
  "fin": "2026-11-08T18:00:00.000Z",
  "estado": "cancelada",
  "creadaEn": "2026-11-04T16:03:05.872Z"
}
```

V6. Comprobar la cancelación

La reserva se conserva en el historial con estado `cancelada` y el horario queda libre.

Petición

```
GET /api/v1/reservas/11
```

Respuesta · HTTP 200

```
{
  "id": 11,
  "usuarioId": 4,
  "recursoId": 1,
  "inicio": "2026-11-08T16:00:00.000Z",
  "fin": "2026-11-08T18:00:00.000Z",
  "estado": "cancelada",
  "creadaEn": "2026-11-04T16:03:05.872Z"
}
```

V7. Error de validación: falta "fin"

Petición

```
POST /api/v1/reservas
{
  "recursoId": 1,
  "inicio": "2026-11-07T16:00:00.000Z"
}
```

Respuesta · HTTP 400

```
{
  "error": {
    "codigo": "VALIDACION_FALLIDA",
    "mensaje": "Los datos enviados no son válidos.",
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-04T16:03:05.907Z",
    "idPeticion": "req-fe9dfffe",
    "detalles": [
      {
        "campo": "fin",
        "problema": "Es obligatorio"
      }
    ]
  }
}
```

V8. Reserva inexistente

Petición

```
GET /api/v1/reservas/999999
```

Respuesta · HTTP 404

```
{
  "error": {
    "codigo": "NO_ENCONTRADO",
    "mensaje": "La reserva no existe.",
    "ruta": "/api/v1/reservas/999999",
    "fecha": "2026-11-04T16:03:05.910Z",
    "idPeticion": "req-c73eb891"
  }
}
```

3.3 Errores transversales

X1. Ruta inexistente

Una ruta que no existe recibe el mismo formato de error que un recurso inexistente.

Petición

```
GET /api/v1/cosas
```

Respuesta · HTTP 404

```
{
  "error": {
    "codigo": "NO_ENCONTRADO",
    "mensaje": "La ruta solicitada no existe.",
    "ruta": "/api/v1/cosas",
    "fecha": "2026-11-04T16:03:05.913Z",
    "idPeticion": "req-77914c7a"
  }
}
```

X2. Petición sin token

Sin token, el Gateway rechaza la petición antes de llegar al servicio.

Petición

```
GET /api/v1/recursos
```

Respuesta · HTTP 401

```
{
  "error": {
    "codigo": "TOKEN_INVALIDO",
    "mensaje": "Falta el token de acceso o no es válido.",
    "ruta": "/api/v1/recursos",
    "fecha": "2026-11-04T16:03:05.915Z",
    "idPeticion": "req-fbe39806"
  }
}
```

X3. Error interno (500)

Para comprobar el manejo de 500 se detuvo PostgreSQL unos segundos y se consultó un recurso. El cliente recibe un mensaje genérico, sin datos internos:

```
{
  "error": {
    "codigo": "ERROR_INTERNO",
    "mensaje": "Ocurrió un error inesperado. Intenta de nuevo más tarde.",
    "ruta": "/api/v1/recursos/1",
    "fecha": "2026-11-04T16:03:30.604Z",
    "idPeticion": "req-f73ce06c"
  }
}
```

El detalle real (`connect ECONNREFUSED 127.0.0.1:5432`) solo queda en el log del servidor, junto con el mismo `idPeticion`. Al volver a encender PostgreSQL, la misma consulta respondió 200 sin reiniciar los servicios. La prueba automática un `error inesperado no revela detalles internos` (test/unit.test.js) verifica el mismo comportamiento.

3.4 Los datos son persistentes

En Recursos, las capturas 3 a 5 trabajan sobre el mismo registro creado en la captura 1. En Reservas, la consulta directa a la base después de la secuencia muestra la reserva modificada y cancelada:

```
reservas_db=> SELECT id, usuario_id, recurso_id, inicio, fin, estado FROM reservas WHERE id =
11;
 id | usuario_id | recurso_id |         inicio         |          fin           |  estado
----+------------+------------+------------------------+------------------------+-----------
 11 |          4 |          1 | 2026-11-08 16:00:00+00 | 2026-11-08 18:00:00+00 | cancelada
(1 row)
```

3.5 Colección de Postman

La colección `postman/PrestaLab.postman_collection.json` tiene **61 peticiones** organizadas en 8 carpetas (autenticación, usuarios, inventario, reservas, préstamos e incidencias, notificaciones, reportes y errores). Inicia sesión sola cuando falta el token o ya venció y encadena los ids con variables, así que se puede correr completa. Resultado de ejecutarla con Newman (reporte en `evidencias/reporte-postman.html`):

Concepto	Ejecutados	Fallidos
Iteraciones	1	0
Peticiones	61	0
Scripts de prueba	61	0
Scripts previos a la petición	6	0
Aserciones	91	0

Además, `npm test` ejecuta 57 pruebas automáticas (todas pasan) y `npm run contrato` confirma que 20 de 20 respuestas reales cumplen los esquemas de `docs/openapi.yaml`.

4. Cambios respecto a la entrega anterior

La arquitectura, el contrato de la API (33 endpoints) y el modelo de datos (10 tablas en 4 bases) **no cambiaron**. Los cambios de esta semana están en las herramientas de prueba y la documentación:

- **Colección de Postman con inicio de sesión automático.** Antes había que correr primero la carpeta «1. Autenticación»; ahora cada petición revisa si tiene el token de su cuenta y, si falta o venció, inicia sesión sola. «Crear recurso» genera su propio código de inventario.
- **Duración del token documentada:** `.env.example` incluye `JWT_EXPIRA_SEGUNDOS` (por defecto 3600), que el sistema ya permitía configurar.
- **Documentación reorganizada:** el README quedó como guía paso a paso de instalación y ejecución; la parte técnica pasó a `docs/arquitectura.md` y la bitácora a `docs/cambios.md`.
- **Contrato sin marca de borrador:** `docs/openapi.yaml` pasa de `1.0.0-borrador` a `1.0.0`. Los endpoints y esquemas son los mismos.
- **Evidencias repetibles:** `npm run evidencias` (`scripts/evidencias-crud.js`) repite la secuencia de Reservas y errores de la sección 3.

5. Criterios de aceptación

Criterio	Estado	Dónde se comprueba
Proyecto backend configurado	Cumplido	Sección 1
Estructura por rutas, controladores, servicios y acceso a datos	Cumplido	Sección 1.1
Variables de entorno para configuración sensible	Cumplido	Sección 1.2 · .env.example
Conexión funcional con la base de datos	Cumplido	Secciones 1.3 y 3.4
CRUD completo de al menos 2 recursos	Cumplido	Sección 2 · capturas 1 a 7 y evidencias V
Validaciones básicas	Cumplido	Captura 6 y evidencia V7
Manejo de 404 y 500	Cumplido	Captura 7 y evidencias V8, X1 y X3
Colección inicial de Postman	Cumplido	Sección 3.5 · postman/
README de ejecución	Cumplido	README.md
El proyecto se ejecuta siguiendo el README	Cumplido	Sección 1.3
Los endpoints funcionan contra datos persistentes	Cumplido	Sección 3.4
El código no está concentrado en un solo archivo	Cumplido	Sección 1.1
Los cambios respecto al diseño están documentados	Cumplido	Sección 4 · Anexo A · docs/cambios.md

Referencias

- OpenJS Foundation. (2025). Express 5.x API Reference. https://expressjs.com/en/5x/api.html
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
- Postman, Inc. (2025). Newman: command-line collection runner. https://learning.postman.com/docs/collections/using-newman-cli/
- Richardson, C. (2018). Microservices Patterns. Manning Publications.

Anexo A. Bitácora de cambios

Semana	Cambio realizado	Motivo	Responsable(s)	Evidencia
1	Versión inicial de la propuesta	Primera entrega	Equipo	Documento de la Entrega 1
2	Estado administrativo separado de la disponibilidad	Evitar responsabilidades duplicadas entre Inventario y Reservas	Equipo	Documento de la Entrega 2
2	10 RF, 9 RNF, matriz de permisos, servicios y diagrama	Contenido de la Entrega 2	Equipo	Documento de la Entrega 2
3	33 endpoints REST y rutas internas; formato único de errores	Contenido de la Entrega 3	Equipo	Entrega 3 y openapi.yaml
3–4	Recurso incluye tipo y encargadoId; rutas internas I-05 e I-06; códigos 422 LABORATORIO_NO_VALIDO y CATEGORIA_NO_VALIDA	Necesidades encontradas al implementar	Equipo	openapi.yaml y Entrega 3 actualizados
4	Tabla roles con llave foránea desde usuarios	La entidad Rol no tenía tabla propia	Equipo	Entrega 4, evidencia C8
4	Datos de prueba en SQL con fechas relativas	Datos reproducibles que no caducan	Equipo	002_datos_prueba.sql
4	Restricciones CHECK con nombre explícito	Mensajes de error claros	Equipo	001_esquema.sql
4	Instalador SQL con rutas relativas (\ir)	Fallaba desde otra carpeta	Equipo	db/instalar.sql
4	Diagrama ER y diccionario generados desde PostgreSQL	Que la documentación no difiera de la base real	Equipo	extraer-modelo.js, generar-er.py
5	Colección de Postman con inicio de sesión automático; código de inventario propio en «Crear recurso»	Las peticiones sueltas respondían 401	Equipo	postman/, sección 4
5	JWT_EXPIRA_SEGUNDOS en .env.example	Documentar una opción que ya existía	Equipo	.env.example
5	README paso a paso; documentación técnica en docs/arquitectura.md y docs/cambios.md	Pedido de la Entrega 5 (README de ejecución)	Equipo	README.md, docs/
5	openapi.yaml versión 1.0.0 (sin «borrador»)	El contrato ya está implementado	Equipo	docs/openapi.yaml
5	Recursos y Reservas como CRUD principales; eliminar = baja lógica	Pedido de la Entrega 5 sin perder historial	Equipo	Sección 2
5	Evidencias de CRUD, errores 400, 404 y 500; script npm run evidencias	Evidencias de la Entrega 5, repetibles en la defensa	Equipo	Sección 3, entregas/entrega-5/